UADE

Facultad de Ingeniería

Trabajo Práctico Obligatorio

Sistemas Operativos

Seis ejercicios en C y Bash sobre la plataforma Red Hat

Segundo cuatrimestre de 2026 · Materia 3.4.075

Docente: José Santiago Franco

Fecha del informe: 2 de noviembre de 2026

Integrantes

Nombre y apellido	Legajo
Javier Candalaft	1244364
Marcos Silva Sapia	1242994
Gadiel Aaron Gerez	1239414
Gianluca Marcellino	1239133

Unidad 1 | Llamadas al sistema

Problema y objetivo

Una aplicación necesita mostrar un mensaje en la terminal. Se busca explicar cómo el pedido de escritura llega al sistema operativo y observar sus argumentos y su resultado, en lugar de limitarse a ver el texto final.

Fundamento teórico

Un programa ejecutado en modo usuario no controla directamente los dispositivos. La interfaz de llamadas al sistema permite solicitar servicios al kernel. En este ejercicio, write recibe un descriptor de archivo, una dirección de memoria y una cantidad de bytes. El descriptor 1 identifica la salida estándar. El wrapper de libc prepara la transición al kernel y devuelve el resultado de la operación.

Implementación y reproducción

unidad-1.c solicita escribir 25 bytes: el mensaje Hola Sistemas Operativos seguido de un salto de línea. El script vigente compila con gcc -O0 -static y luego usa objdump -d para desensamblar el ejecutable. El enlazado estático requiere que las bibliotecas estáticas correspondientes estén disponibles en el laboratorio.

```
cd unidad-1
bash compilar.sh
strace -e trace=write ./unidad-1
```

La traza esperada es write(1, "Hola Sistemas Operativos\n", 25) = 25. El retorno expresa los bytes escritos. El programa es mínimo: no comprueba errores ni escrituras parciales.

Resultados

La primera captura muestra ensamblador x86-64 generado por GCC y una llamada a write. La segunda muestra la traza de una escritura de 25 bytes y finalización con código 0. Ambas provienen de la consola Red Hat.

El ensamblador permite identificar cómo se preparan los argumentos y cómo se llama a write. Puede obtenerse desde el código C con gcc -S o mediante el desensamblado del ejecutable con objdump -d. La primera captura ilustra el primer procedimiento; el script incluido en el anexo utiliza el segundo.

Conclusión

La traza de strace permite relacionar una instrucción del programa con un servicio concreto del kernel. El proceso solicita la escritura desde modo usuario y recibe como resultado la cantidad de bytes escritos.

Evidencia de la Unidad 1

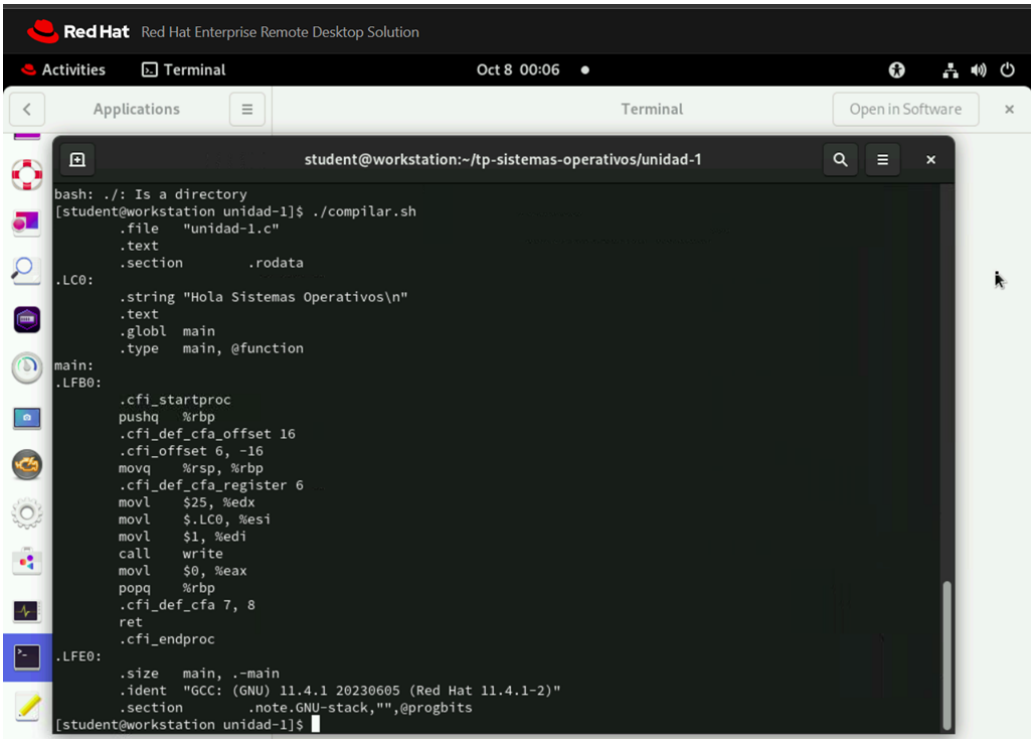


Figura 1. Compilación y listado ensamblador en Red Hat. Fuente: unidad-1/Evidencia U1.png.

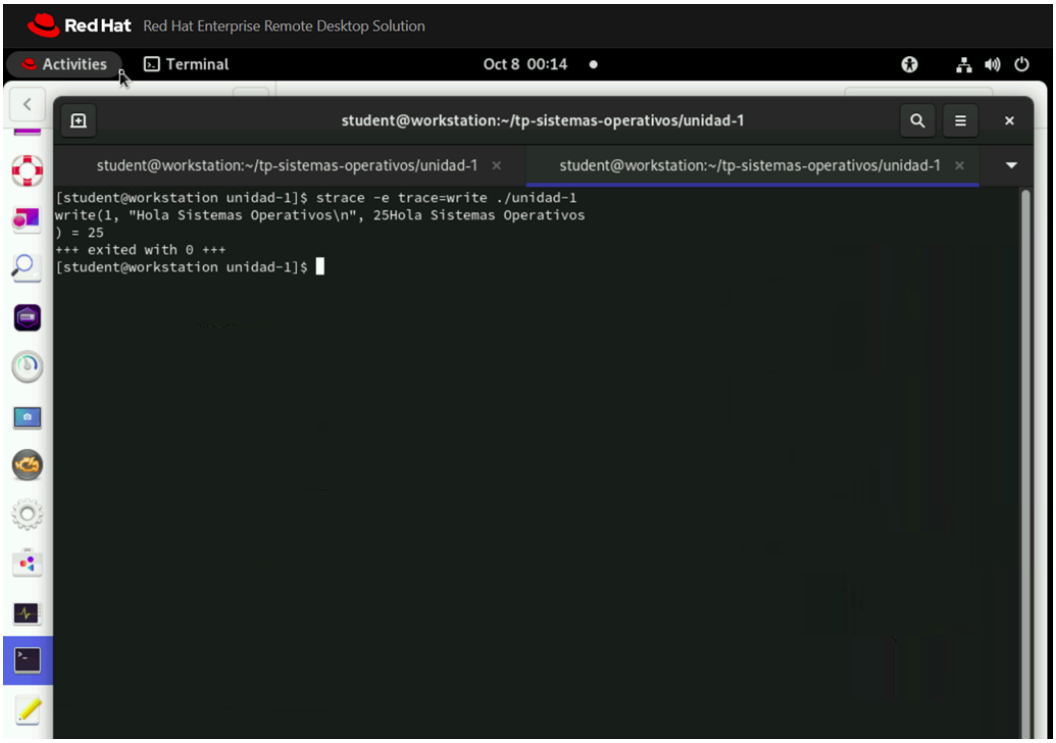


Figura 2. Traza de write: solicitud de 25 bytes, retorno 25 y salida con código 0. Fuente: unidad-1/Evidencia U1 - 2.png.

Unidad 2 | Paralelismo y pseudoparalelismo

Problema y objetivo

Una computadora debe completar diez tareas independientes. Cada una calcula la suma de los números de 1 a 1.000.000.000. Se compara el tiempo total cuando las tareas se ejecutan una por una, concurrentemente en una CPU lógica o concurrentemente con acceso a todas las CPU permitidas.

Fundamento teórico

La concurrencia permite mantener varios procesos en progreso. En una única CPU lógica, el planificador alterna su ejecución: esto produce pseudoparalelismo. Con varias CPU disponibles puede haber ejecución simultánea. El tiempo real depende también de las cuotas, la carga y la configuración de la máquina virtual.

Implementación

El programa C ejecuta la suma con unsigned long long e imprime 500000000500000000. El script compila sin optimizaciones, obtiene la afinidad permitida mediante taskset y toma la primera CPU de esa lista para las dos primeras modalidades. Cada modalidad ejecuta exactamente diez veces el mismo programa.

La modalidad secuencial espera cada proceso antes de iniciar el siguiente. Las otras dos usan & para iniciar procesos en segundo plano y wait para esperar cada PID. Si alguno falla, el script interrumpe el reporte. Con una sola CPU permitida se omite la modalidad paralela.

```
cd unidad-2
bash unidad-2.sh
```

El script comprueba gcc, taskset, date y awk; instala los paquetes faltantes con dnf. La medición con date +%s%N incluye el inicio, cálculo, impresión y espera de procesos, pero no la compilación. El cociente entre tiempo secuencial y paralelo se informa como aceleración.

Análisis de las modalidades

La suma debe producir el mismo resultado numérico en las diez tareas de cada modalidad: 500000000500000000. La pseudoparalela no debería aportar una mejora sustancial para este trabajo de CPU sobre una sola CPU lógica.

El reporte del script presenta los tiempos de cada modalidad y calcula la aceleración como tiempo secuencial dividido por tiempo paralelo. Estos valores dependen de los recursos disponibles: varias CPU lógicas pueden compartir núcleos físicos o estar sujetas a cuotas de uso.

Conclusión

El ejercicio mantiene constante la cantidad de trabajo y cambia la forma de planificarlo. Permite separar concurrencia de paralelismo y discutir por qué iniciar más procesos no garantiza reducir el tiempo total.

Evidencia de la Unidad 2 | Ejecución

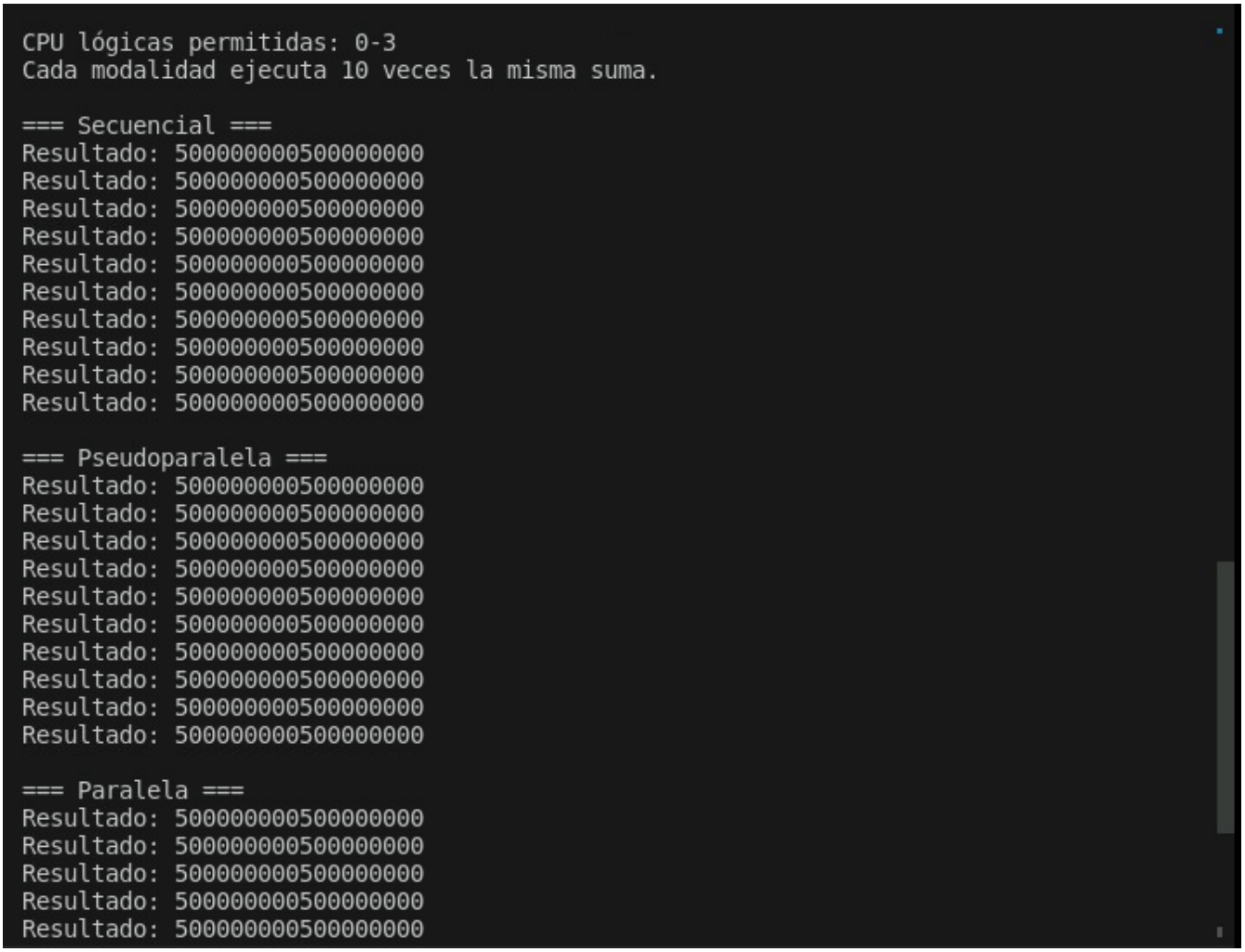


Figura 3. CPU permitidas y resultados de las tareas secuenciales y concurrentes. Fuente: unidad-2/unidad-2-evidencia-1.jpeg. Detalle de la terminal; se omite el marco de escritorio.

La ejecución dispone de cuatro CPU lógicas, numeradas de 0 a 3. La terminal muestra el mismo resultado numérico en las tareas: 500000000500000000. El script mantiene esa carga de trabajo y cambia la afinidad y la concurrencia entre modalidades.

Evidencia de la Unidad 2 | Tiempos

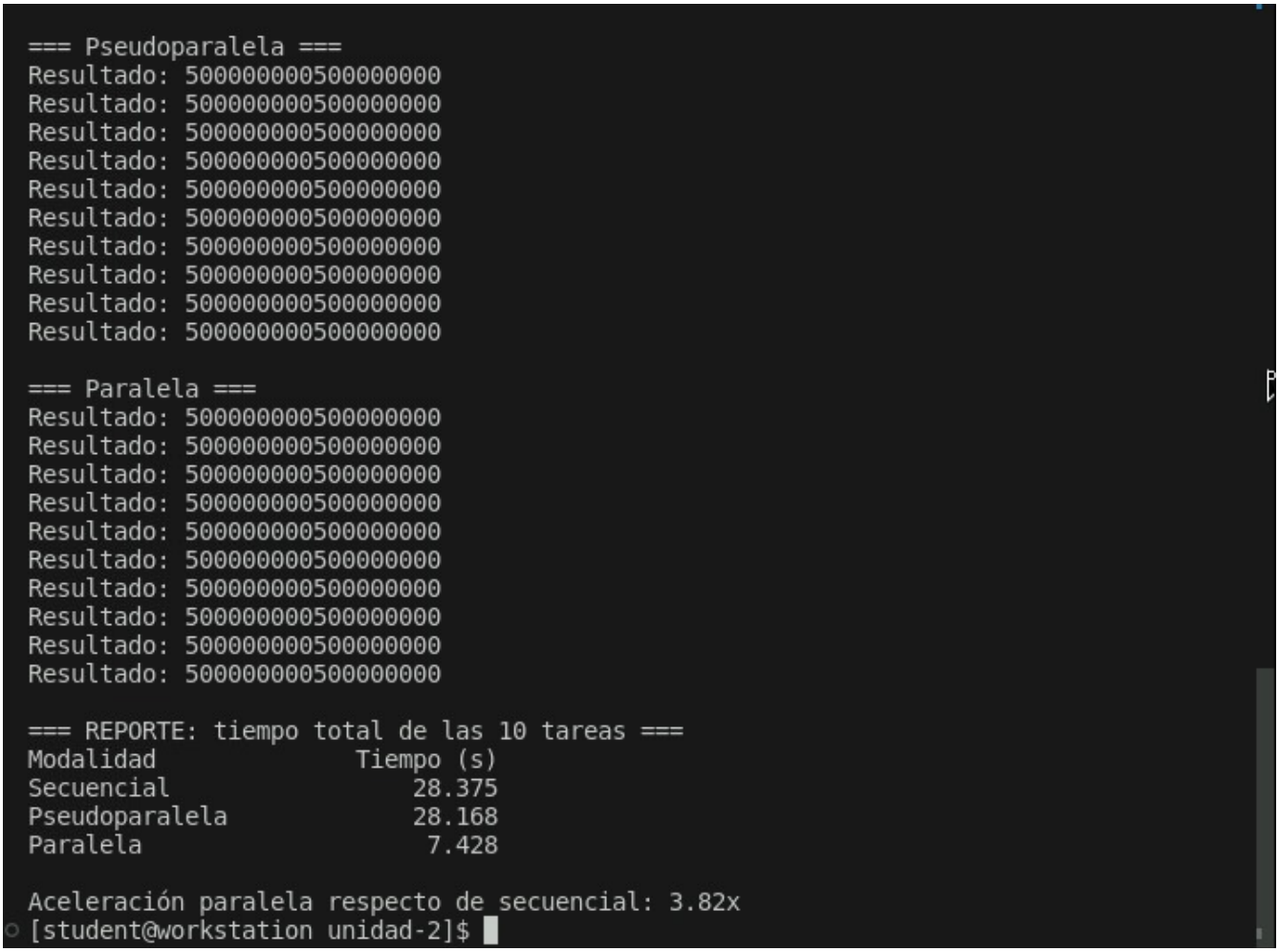


Figura 4. Reporte del tiempo total de diez tareas por modalidad. Fuente: unidad-2/unidad-2-evidencia-2.jpeg. Detalle de la terminal; se omite el marco de escritorio.

Resultados de la ejecución

• Secuencial: 28,375 segundos.

• Pseudoparalela: 28,168 segundos.

• Paralela: 7,428 segundos.

• Aceleración paralela respecto de la secuencial: 3,82 veces.

Los tiempos secuencial y pseudoparalelo son similares, mientras que el acceso concurrente a las cuatro CPU permitidas reduce el tiempo total. La aceleración se calcula como 28,375 / 7,428 y corresponde a esta ejecución concreta.

Unidad 3 | Compilación, carga y memoria

Problema y objetivo

Se plantea el escenario hipotético de un servicio backend que sufre un pico repentino de consumo de memoria. La aplicación necesita reservar 100 MiB y acceder de manera espaciada a esa región. Se estudia cómo se construye el ejecutable, cómo se monitorea su ejecución en segundo plano y cómo se observa su consumo de memoria virtual y residente utilizando herramientas nativas del sistema operativo.

Fundamento teórico

La construcción comprende preprocesamiento, compilación, ensamblado y enlace. Al ejecutar el programa, el sistema crea su espacio de direcciones y prepara la imagen del proceso. Las direcciones virtuales utilizadas por el código se traducen mediante las estructuras de memoria y la MMU. La reserva de un rango de direcciones virtuales y la asignación de páginas físicas no son necesariamente el mismo evento. En la paginación por demanda, el sistema resuelve los accesos a páginas que todavía no están presentes cuando el proceso las necesita.

Implementación

compilar.sh verifica GCC, compila paginacion.c con -Wall -Wextra -O0 y presenta el contexto de la simulación. Para permitir el monitoreo, el script ejecuta el binario en segundo plano (`&`) y captura su PID. El programa solicita 104.857.600 bytes con malloc tras una pausa inicial de tres segundos. Luego escribe un byte cada 4096 posiciones, produciendo 25.600 escrituras separadas, y mantiene la reserva durante diez segundos para permitir su observación.

Mientras el programa retiene la memoria, el script utiliza comandos de la consola de Red Hat (`free`, `ps`, `vmstat`) para auditar el proceso.

```
cd unidad-3
bash compilar.sh
```

Relación con la memoria virtual

Con páginas de 4096 bytes, el recorrido relaciona cada salto del programa con la granularidad de la paginación. Al escribir en páginas todavía no respaldadas por memoria física, el acceso provoca fallos de página que el sistema atiende asignando los recursos correspondientes. La cantidad de fallos depende de la administración de memoria del entorno y no se cuenta en este programa.

La consulta `ps` distingue el tamaño del espacio virtual del proceso (VSZ) de su memoria residente (RSS). Ambos incluyen más componentes que el bloque reservado, como código, bibliotecas y pila. `vmstat 1 3` y `free -h` aportan estadísticas globales; su lectura debe considerar la actividad de otros procesos y el redondeo de las unidades.

La compilación sin -static utiliza el enlace habitual del entorno, normalmente dinámico en Red Hat. Así, el ejercicio conecta la generación del ejecutable con su carga y el acceso a direcciones virtuales durante la ejecución.

Conclusión

La reserva con malloc, el recorrido del bloque y su liberación permiten seguir el uso de memoria desde el programa. El sistema operativo administra el espacio de direcciones y su respaldo físico, mientras que las herramientas de consola permiten observar la memoria residente del proceso durante la reserva. Al terminar el proceso, el sistema libera los recursos que le pertenecían.

Evidencia de la Unidad 3

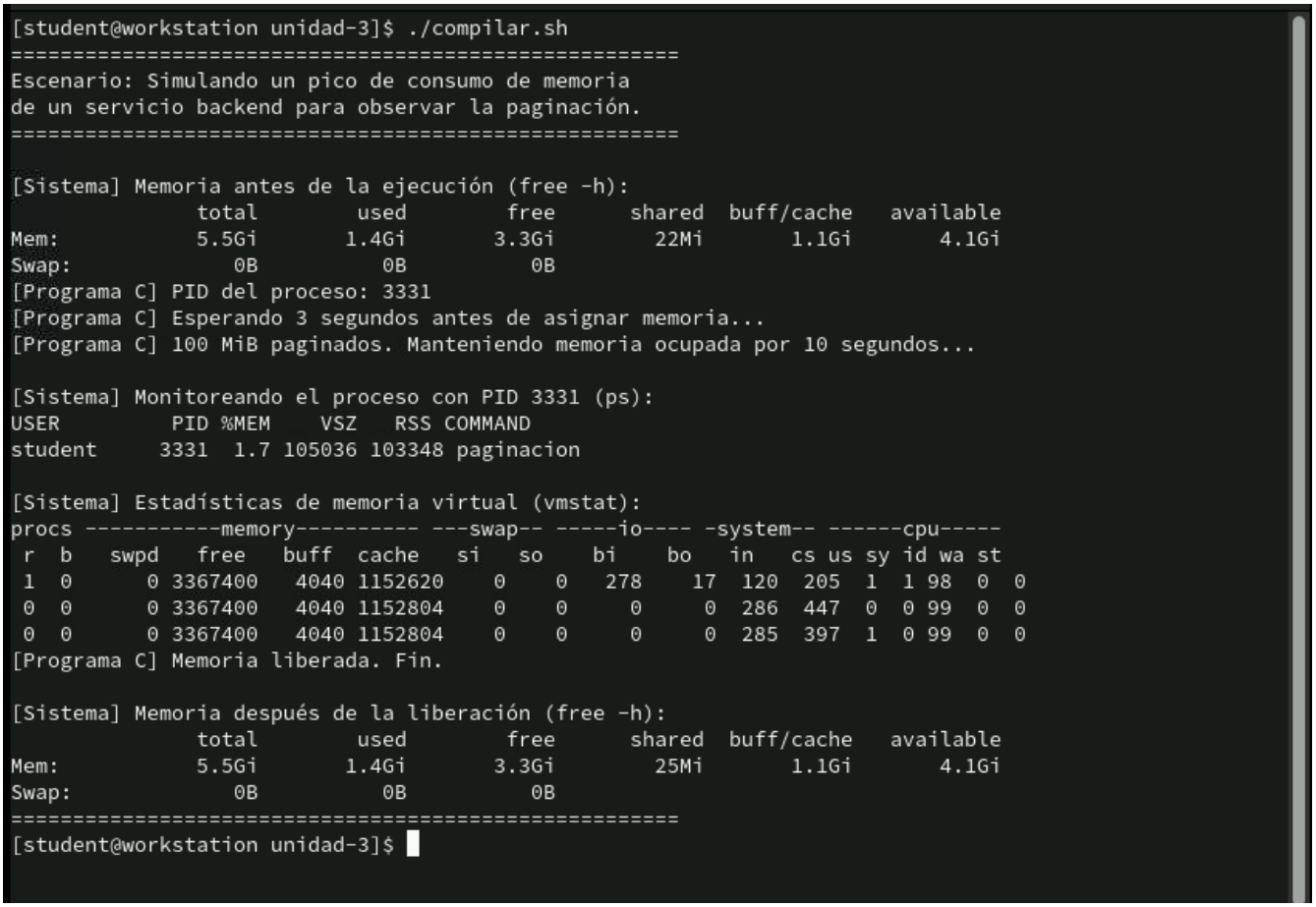


Figura 5. Monitoreo de memoria en Red Hat: ejecución en segundo plano, contraste de memoria VSZ/RSS y estadísticas en tiempo real. Fuente: unidad-3/Evidencia U3.png. Detalle de la terminal; se omite el marco de escritorio.

Resultados observados

La captura muestra el proceso paginacion con PID 3331, VSZ de 105.036 KiB y RSS de 103.348 KiB, aproximadamente 100,9 MiB de memoria residente. Este último valor es coherente con el bloque de 100 MiB utilizado por el programa y el resto de su memoria.

El entorno informa 0 B de swap y las columnas si/so de vmstat permanecen en cero. En esta ejecución no se observa actividad de intercambio con disco. Los valores redondeados de free -h antes y después coinciden; la evidencia directa del consumo durante la reserva es la consulta de RSS realizada con ps.

Unidad 4 | Almacenamiento y sistema de archivos

Problema y objetivo

Un laboratorio dispone de un disco virtual adicional destinado a datos. Se propone crear una partición, formatearla con XFS y montar su contenido en /mnt/datos de forma persistente.

Fundamento teórico

Una partición delimita una región del dispositivo. El formateo crea las estructuras del sistema de archivos. El montaje incorpora ese sistema de archivos al árbol de directorios. Una entrada en /etc/fstab permite declarar su montaje y el UUID identifica el sistema de archivos independientemente del nombre del dispositivo.

Implementación

unidad-4.sh utiliza /dev/vdb como disco y /dev/vdb1 como partición. Envía a fdisk una secuencia de respuestas para crear una partición primaria, ejecuta mkfs.xfs -f, crea /mnt/datos y obtiene el UUID con blkid. Después actualiza la entrada correspondiente en /etc/fstab y ejecuta mount -a y df -h.

El procedimiento está planteado para un disco virtual de laboratorio vacío, identificado previamente y administrado con permisos de root. La secuencia de fdisk supone una tabla de particiones compatible con la creación de una partición primaria. El formateo reemplaza el sistema de archivos del dispositivo seleccionado.

```
cd unidad-4
sudo bash unidad-4.sh
```

Partición, formato y montaje

fdisk define la partición que se utilizará para almacenar los datos. mkfs.xfs crea sus estructuras de sistema de archivos. mkdir -p prepara el directorio de acceso y mount incorpora el volumen al árbol de directorios. Son pasos diferentes: crear una partición no equivale a formatearla, y formatearla no implica que ya esté montada.

La entrada de /etc/fstab usa UUID para identificar el sistema de archivos y /mnt/datos como destino. De esta forma, la configuración de montaje no depende exclusivamente de que el dispositivo mantenga el nombre /dev/vdb1.

Comprobación del montaje

lsblk permite consultar los dispositivos y particiones; blkid muestra el UUID; findmnt /mnt/datos y df -h permiten comprobar el montaje. Estas consultas verifican el estado del sistema independientemente de los mensajes que imprima el script.

El archivo C del anexo es un lanzador que invoca ./unidad-4.sh mediante system. El procedimiento indicado arriba permite ejecutar el mismo script directamente con Bash y permisos de administración.

Conclusión

El ejercicio integra particionado, formato y montaje persistente. El uso de UUID vincula la configuración con el sistema de archivos, mientras que las herramientas de consola permiten consultar cada etapa de la administración del almacenamiento.

Evidencia de la Unidad 4

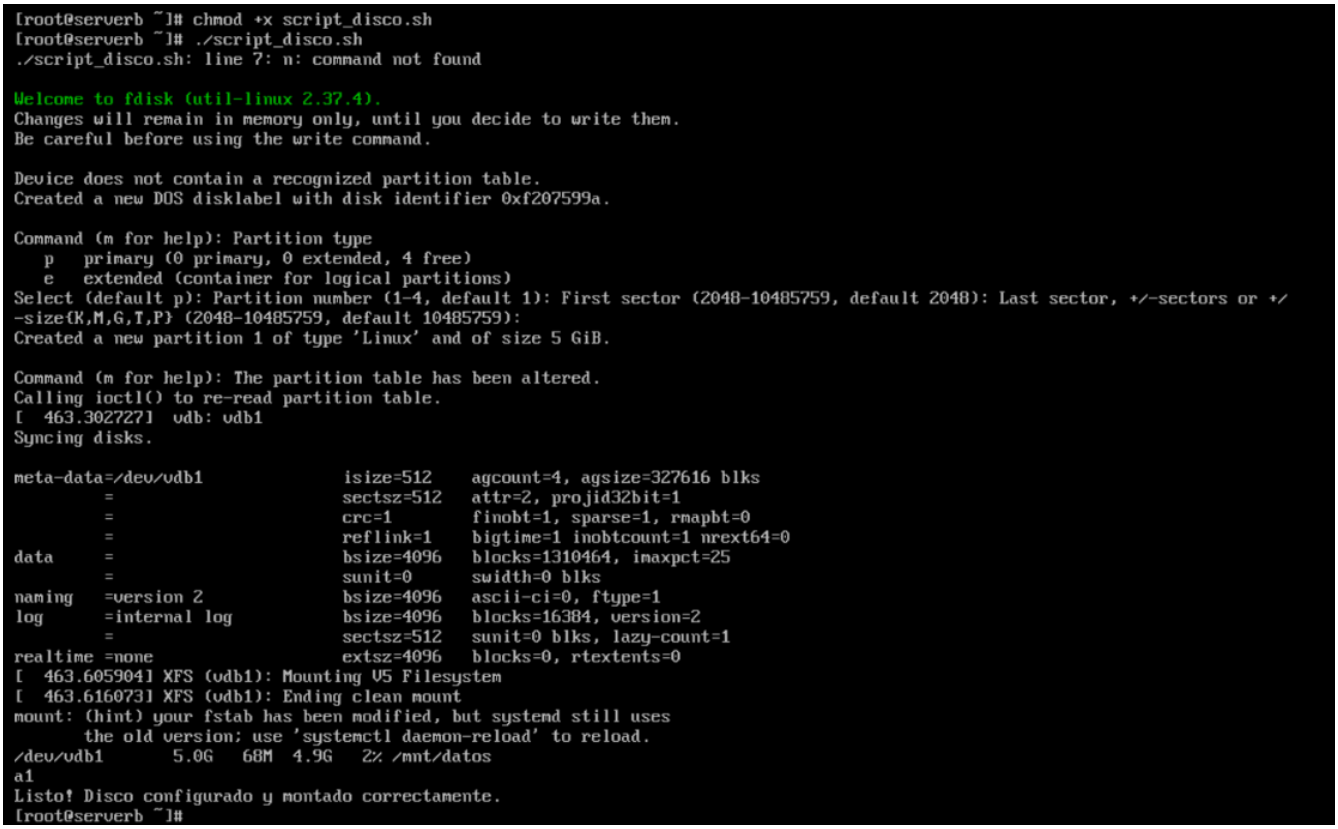


Figura 6. Creación de la partición, formateo XFS y montaje de /dev/vdb1 en /mnt/datos. Fuente: unidad-4/unidad-4.png. Detalle de la terminal; se omite el marco de escritorio.

Resultados observados

La sesión de consola muestra la creación de una partición de 5 GiB, el formateo con XFS y la consulta de espacio del volumen montado. La línea final de df identifica /dev/vdb1 en /mnt/datos, con 5,0 G de capacidad, 68 M utilizados y 4,9 G disponibles.

La configuración de /etc/fstab expresa el montaje para futuros arranques. La captura permite observar el montaje durante esta sesión; la persistencia después de reiniciar se comprueba por separado.

Unidad 5 | Entrada y salida: parte A

Problema y objetivo

Se necesita identificar dispositivos de bloque y carácter y analizar cuánto cambia el costo de leer un archivo al variar el tamaño del búfer. También se observa el contador de interrupciones antes y después de una escritura de disco.

Fundamento teórico

Los archivos especiales de /dev permiten acceder a dispositivos. El número mayor identifica su clase o controlador y el menor distingue dispositivos dentro de esa clase. Las lecturas solicitan al kernel transferir datos al proceso. Un búfer mayor permite solicitar más bytes por operación y reducir la cantidad de invocaciones para recorrer el mismo archivo.

Implementación

El script compila el programa, selecciona el primer disco que coincide con su criterio en lsblk y genera prueba.bin con 2.000.000 de bytes. El programa consulta stat sobre los dispositivos recibidos y recorre el archivo usando búferes de 1, 512, 4096 y 65536 bytes. Mide con CLOCK_MONOTONIC. El script consulta /proc/devices y compara contadores de /proc/interrupts alrededor de una escritura con dd y oflag=direct.

```
cd unidad-5
bash ejecutar.sh
```

Resultados observados

La captura muestra /dev/vda como dispositivo de bloque, mayor 252 y menor 0; /dev/null, /dev/tty y /dev/urandom aparecen como dispositivos de carácter. Los tiempos y conteos visibles se transcriben en la página siguiente.

Análisis de los resultados

El contador del C aumenta solo cuando read devuelve un valor positivo. No incluye la última invocación que devuelve 0 por fin de archivo. Por eso la columna del programa representa lecturas con datos, no todas las llamadas realizadas. Para un búfer de 1 byte hay 2.000.000 de lecturas positivas y una comprobación final de fin de archivo.

La caché de páginas y el orden fijo de las pruebas influyen en los tiempos: no son una medición aislada de velocidad del disco. El contador de interrupciones agrupa coincidencias de varios controladores y puede incluir actividad ajena al ejercicio. No demuestra por sí mismo el uso de DMA ni una correspondencia de una interrupción por transferencia.

Además, bs=1M y count=100 de dd corresponden a 100 MiB (104.857.600 bytes), aunque el mensaje del script diga 100 MB. El archivo de lectura sí contiene 2 MB decimales.

Evidencia de la Unidad 5 | Parte A

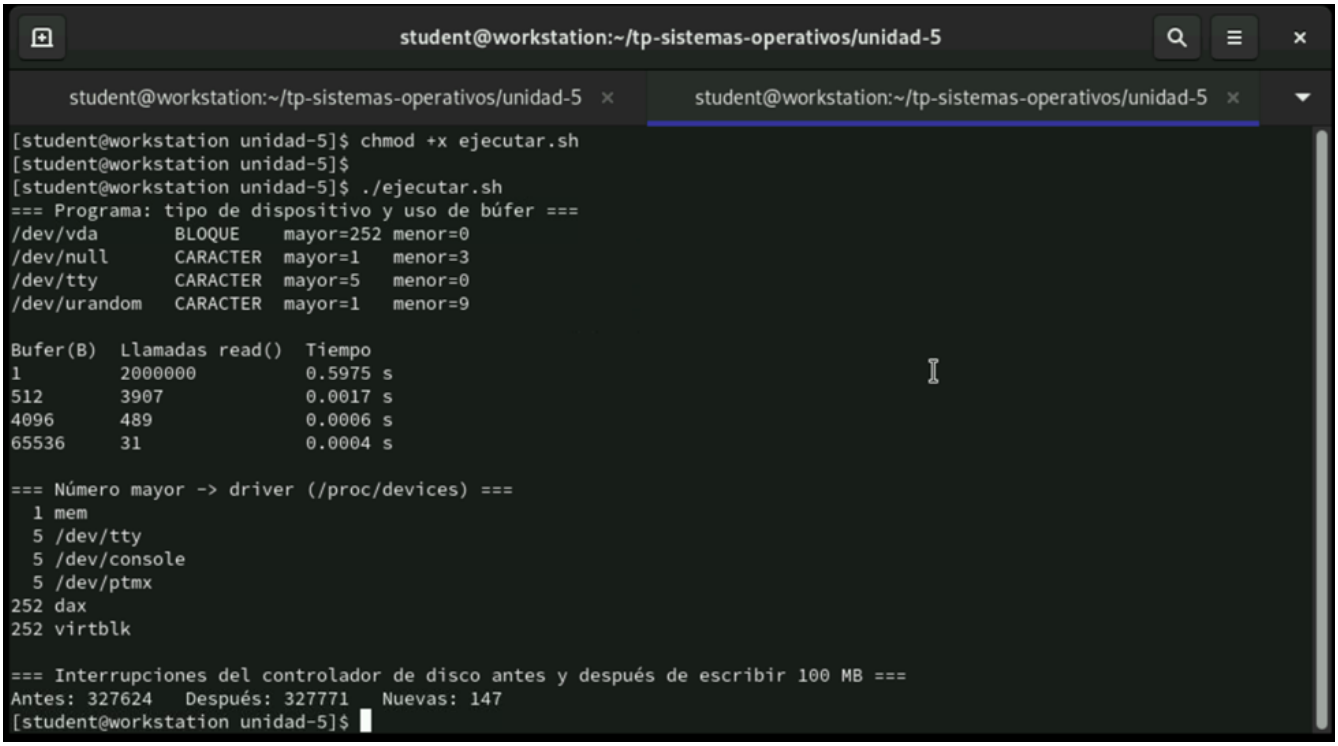


Figura 7. Dispositivos, lecturas con distintos búferes e interrupciones en Red Hat. Fuente: unidad-5/Evidencia U5-Parte A.png. Detalle de la terminal; se omite el marco de escritorio.

Transcripción de la captura

• Búfer de 1 byte: 2.000.000 de lecturas positivas; 0,5975 s.

• Búfer de 512 bytes: 3.907 lecturas positivas; 0,0017 s.

• Búfer de 4096 bytes: 489 lecturas positivas; 0,0006 s.

• Búfer de 65536 bytes: 31 lecturas positivas; 0,0004 s.

• Interrupciones: 327.624 antes, 327.771 después; diferencia de 147.

Conclusión

La evidencia muestra que agrupar los bytes reduce fuertemente la cantidad de lecturas y el tiempo observado en esta ejecución. La explicación se relaciona con el costo de las operaciones de entrada/salida y el uso de búferes; la magnitud de la mejora no debe generalizarse a todos los equipos o cargas.

Unidad 5 | Parte B: drivers y fallos

Problema y relación con la unidad

Un componente interpreta una configuración con veinte campos. Una actualización solicita un campo adicional. El ejercicio analiza qué sucede al acceder a ese campo desde un proceso de usuario y relaciona el resultado con el impacto de un fallo en un componente que se ejecuta dentro del kernel.

Caso CrowdStrike

El 19 de julio de 2024, una actualización de contenido de Falcon para Windows provocó fallos de sistema. El análisis de CrowdStrike explica la discrepancia entre veinte entradas suministradas al intérprete y el acceso a una entrada número veintiuno. La falta de una comprobación de límites permitió una lectura fuera del arreglo. El caso relaciona la validación de datos con la importancia del contexto de ejecución de los controladores. Fuente: CrowdStrike, análisis de causa raíz del Channel File 291 (2024).

Implementación de la simulación

crowdstrike.c consulta el tamaño de página con sysconf, reserva dos páginas con mmap y protege la segunda con mprotect y PROT_NONE. Coloca veinte punteros al final de la primera página. De esta manera, el acceso al puntero veintiuno cae en la página protegida. Esta disposición fuerza un fallo de acceso para la demostración; una lectura fuera de límites en un programa cualquiera no siempre produce una caída inmediata.

Con el argumento --validar, el programa rechaza una solicitud mayor que veinte antes de recorrer los campos. La simulación se ejecuta en modo usuario y no carga un driver ni modifica el kernel.

```
cd unidad-5
bash crowdstrike.sh
```

El script compila el programa, consulta los módulos con lsmod y modinfo, ejecuta el caso de veintiún campos, muestra su estado de salida y consulta uptime y dmesg. Luego ejecuta la variante con validación.

Comportamiento de los casos

• ./crowdstrike 20: recorre los veinte campos y termina normalmente.

• ./crowdstrike 21: intenta leer la página protegida; en el entorno Linux previsto se espera SIGSEGV, representada habitualmente por Bash como estado 139.

• ./crowdstrike 21 --validar: rechaza la cantidad solicitada y devuelve estado 1 sin realizar el acceso inválido.

La señal concreta y los registros disponibles dependen del entorno. La consulta de dmesg puede requerir permisos de administración. El código de salida permite distinguir la terminación normal, el rechazo controlado y la terminación por una señal.

Conclusión

En modo usuario, el sistema puede terminar el proceso que realiza el acceso inválido y continuar atendiendo otros procesos. Un fallo dentro del kernel puede comprometer al sistema completo, como ocurrió en el caso analizado. La demostración representa el control de límites y el aislamiento de procesos; no reproduce el componente Falcon ni una caída real del kernel.

Evidencia de la Unidad 5 | Parte B

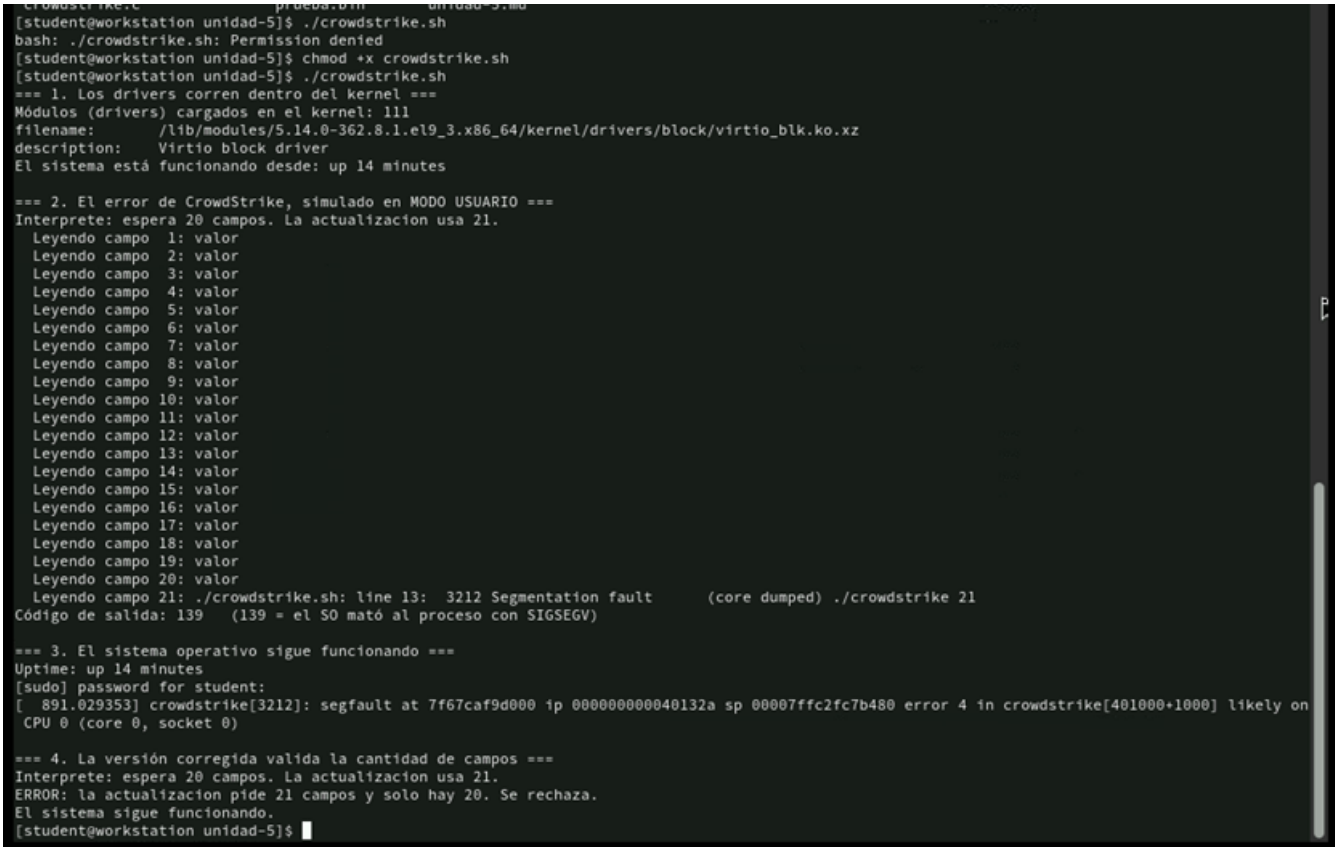


Figura 8. Ejecución de la simulación CrowdStrike en Red Hat y comparación con la validación de campos. Fuente: unidad-5/Evidencia U5-Parte B.png. Detalle de la terminal; se omite el marco de escritorio.

Resultados observados

La sesión informa 111 módulos cargados y muestra información de virtio_blk. Durante la simulación, los veinte primeros campos se leen normalmente. El acceso al campo veintiuno provoca Segmentation fault y el script informa el código de salida 139.

Después del fallo, uptime muestra que el sistema sigue activo y dmesg registra el acceso inválido del proceso crowdstrike. La ejecución con --validar rechaza la solicitud de veintiún campos e imprime que el sistema sigue funcionando. La comparación muestra el aislamiento del proceso y el efecto de controlar la cantidad de campos antes de leerlos.

Unidad 6 | Seguridad y exposición de memoria

Problema y objetivo

Una aplicación permite consultar el nombre de usuario y prohíbe consultar una contraseña ficticia. El solicitante elige el campo y la longitud. Se demuestra que bloquear el acceso directo al campo privado no alcanza cuando una longitud excesiva permite incluir sus bytes en otra respuesta.

Fundamento teórico

La protección de memoria del sistema operativo separa espacios de procesos, pero no interpreta las reglas de confidencialidad de cada aplicación. Si ambos campos están en memoria accesible para el proceso, es responsabilidad del programa limitar qué bytes puede entregar.

Implementación

El array char datos[2][16] contiene usuario y clave123, con terminadores y relleno de ceros. Son datos de demostración. El programa rechaza el campo 1, acepta el campo 0 y permite longitudes de hasta sizeof(datos), es decir, 32 bytes. write recibe la dirección del array completo y la longitud solicitada. Como no interpreta cadenas, no se detiene ante los bytes nulos.

```
cd unidad-6
bash unidad-6.sh
# Una vez compilado:
printf '0\n32\n' | ./unidad-6 | od -An -tx1c
strace -e trace=write ./unidad-6
```

Casos y corrección propuesta

• Campo 1: acceso rechazado antes de pedir longitud.

• Campo 0 y longitud 16: consulta de la primera fila completa.

• Campo 0 y longitud 32: respuesta con ambos campos.

• Campo 0 y longitud 33: rechazo por superar el array completo.

La corrección propuesta es comparar con sizeof(datos[0]), que limita el acceso a los 16 bytes del campo permitido. El código del repositorio conserva intencionalmente la validación vulnerable. De esta manera, una solicitud de 32 bytes se rechazaría antes de realizar la escritura.

Relación con Heartbleed

OWASP describe Heartbleed (CVE-2014-0160) como una falta de validación de límites antes de copiar datos, que permitió exponer memoria de OpenSSL. La analogía es devolver más información que la autorizada por confiar en una longitud. Este ejercicio permanece dentro del array de 32 bytes; no reproduce TLS, no lee otros procesos ni accede fuera del objeto completo. Fuente: OWASP, Heartbleed Bug, consultada el 02/11/2026.

Evidencia de la Unidad 6

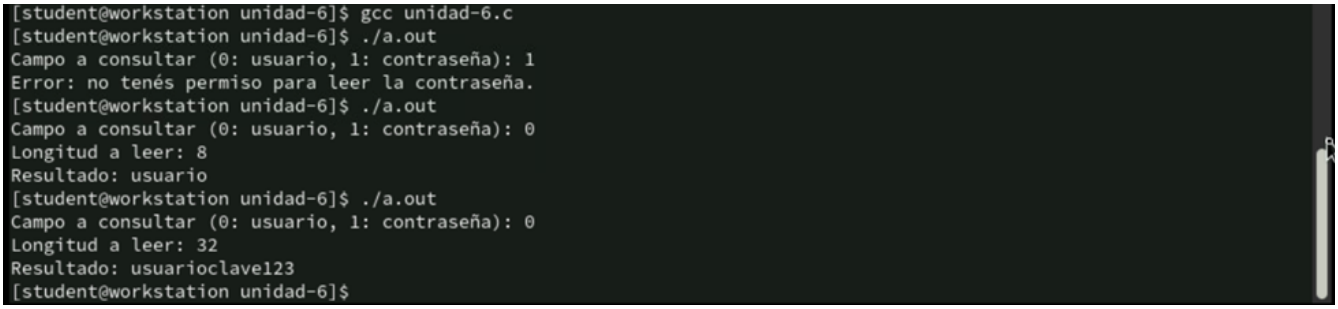


Figura 9. Rechazo del campo privado, consulta normal y exposición mediante longitud 32. Fuente: unidad-6/Evidencia U6.png. Detalle de la terminal; se omite el marco de escritorio.

Lectura de la evidencia

La imagen original registra la instalación de GCC. El detalle reproducido muestra la compilación manual con gcc unidad-6.c y tres ejecuciones de ./a.out. Primero se rechaza el campo 1. Después se solicita el campo 0 con longitud 8 y se ve usuario. Finalmente, con longitud 32 se observa usuarioclave123; los ceros de relleno no son visibles en la terminal.

Los tres casos permiten comparar el control sobre el campo solicitado con el control sobre la longitud: el acceso directo se rechaza, pero una consulta al campo autorizado puede abarcar también el dato privado.

Conclusión

La confidencialidad depende tanto del campo autorizado como de la extensión del acceso. Un control correcto sobre el selector puede quedar anulado por otro control demasiado amplio sobre la longitud. La operación puede ser válida para el kernel y, al mismo tiempo, violar la regla de seguridad de la aplicación.

Conclusiones generales

Los seis ejercicios relacionan los conceptos de Sistemas Operativos con programas C y herramientas de consola de Red Hat. Las llamadas al sistema permiten solicitar servicios al kernel; la planificación determina cómo se distribuye el tiempo de CPU; y la memoria virtual ofrece a cada proceso un espacio de direcciones sobre el que trabaja la aplicación.

La administración de almacenamiento conecta dispositivos, particiones, sistemas de archivos y puntos de montaje. En entrada y salida, el tamaño del búfer influye sobre la cantidad de operaciones necesarias para procesar los mismos datos. El caso de los controladores permite comparar las consecuencias de un fallo según el modo de ejecución. Finalmente, el ejercicio de seguridad muestra que la protección del sistema operativo debe complementarse con controles adecuados dentro del programa.

Bash permite organizar estos procedimientos: compilar, ejecutar procesos, esperar su finalización, medir tiempos y consultar el estado del sistema. C permite observar de forma directa operaciones de memoria y entrada/salida. Su uso conjunto vincula la explicación teórica con el funcionamiento de cada ejercicio.

Referencias

• UADE. Sistemas Operativos: consigna del Trabajo Práctico Obligatorio y material de la asignatura. Segundo cuatrimestre de 2026.

• Código fuente y capturas de los ejercicios incluidos en este trabajo.

• OWASP Foundation. Heartbleed Bug. https://community.owasp.org/vulnerabilities/Heartbleed_Bug (consulta: 02/11/2026).

• CrowdStrike. Executive Summary: Root Cause Analysis - Channel File 291, 6 de agosto de 2024. https://www.crowdstrike.com/wp-content/uploads/2024/08/Executive-Summary_Root-Cause-Analysis_Channel-File-291.pdf (consulta: 02/11/2026).

Anexo | Código fuente

unidad-1/compilar.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #!/bin/bash
 2  set -e
 3
 4  cd -- "$(dirname -- "$0")"
 5
 6  # Insalar GCC si no está instalado.
 7  if ! command -v gcc &> /dev/null; then
 8
 9      echo "GCC no está instalado. Instalando..."
10
11      sudo dnf install -y gcc
12
13  fi
14  # Compilar el programa para ejecutarlo en Red Hat.
15  gcc -O0 -static unidad-1.c -o unidad-1
16
17  # Generar ensamblador desde el código C, sin optimizaciones.
18  objdump -d unidad-1 > unidad-1.s
19
20  # Mostrar el ensamblador generado.

21  cat ./unidad-1.s
```

Anexo | Código fuente

unidad-1/unidad-1.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
1  #include <unistd.h>
2
3  int main(void) {
4      write(1, "Hola Sistemas Operativos\n", 25);
5      return 0;
6  }
```

Anexo | Código fuente

unidad-1/unidad-1.s

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

Listado ensamblador incluido en el repositorio: plataforma macOS ARM64, llamada a printf.

```
 1      .build_version macos, 27, 0 sdk_version 27, 0
 2      .section    __TEXT,__text,regular,pure_instructions
 3      .globl  _main                           ; -- Begin function main
 4      .p2align    2
 5  _main:                                      ; @main
 6      .cfi_startproc
 7  ; %bb.0:
 8      sub sp, sp, #32
 9      stp x29, x30, [sp, #16]             ; 16-byte Folded Spill
10      add x29, sp, #16
11      .cfi_def_cfa w29, 16
12      .cfi_offset w30, -8
13      .cfi_offset w29, -16
14      mov w8, #0                          ; =0x0
15      str w8, [sp, #8]                    ; 4-byte Folded Spill
16      stur    wzr, [x29, #-4]
17      adrp    x0, l_.str@PAGE
18      add x0, x0, l_.str@PAGEOFF
19      bl  _printf
20      ldr w0, [sp, #8]                    ; 4-byte Folded Reload

21      ldp x29, x30, [sp, #16]             ; 16-byte Folded Reload
22      add sp, sp, #32
23      ret
24      .cfi_endproc
25                                          ; -- End function
26      .section    __TEXT,__cstring,cstring_literals
27  l_.str:                                 ; @.str
28      .asciz  "Hola Sistemas Operativos\n"
29
30  .subsections_via_symbols
```

Anexo | Código fuente

unidad-2/unidad-2.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #include <stdio.h>
 2
 3  int main(void)
 4  {
 5      unsigned long long suma = 0;
 6
 7      for (unsigned long long i = 1; i <= 1000000000ULL; i++) {
 8          suma += i;
 9      }
10
11      printf("Resultado: %llu\n", suma);
12      return 0;
13  }
```

Anexo | Código fuente

unidad-2/unidad-2.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
#!/bin/bash
set -euo pipefail
cd -- "$(dirname -- "$0")"

# Instalar sólo los paquetes correspondientes a herramientas faltantes.
paquetes=()
command -v gcc >/dev/null 2>&1 || paquetes+=(gcc)
command -v taskset >/dev/null 2>&1 || paquetes+=(util-linux)
command -v date >/dev/null 2>&1 || paquetes+=(coreutils)
command -v awk >/dev/null 2>&1 || paquetes+=(gawk)

if (( ${#paquetes[@]} > 0 )); then
    echo "Instalando herramientas faltantes: ${paquetes[*]}"
    if (( EUID == 0 )); then
        dnf install -y "${paquetes[@]}"
    else
        sudo dnf install -y "${paquetes[@]}"
    fi
fi


# Sin optimizaciones: conservar el bucle de suma.
gcc -Wall -Wextra -O0 unidad-2.c -o unidad-2

# Respetar las CPU que el laboratorio permite usar a este proceso.
cpus=$(LC_ALL=C taskset -pc "$$")
cpus=${cpus##*: }
primera_cpu=${cpus%%[,-]*}

echo "CPU lógicas permitidas: $cpus"
echo "Cada modalidad ejecuta 10 veces la misma suma."

medir() {
    local modo=$1 afinidad=$2 inicio fin pid
    local pids=()
    echo
    echo "=== $modo ==="
    inicio=$(date +%s%N)

    for (( tarea=1; tarea<=10; tarea++ )); do
        if [[ "$modo" == "Secuencial" ]]; then

            taskset -c "$afinidad" ./unidad-2
        else
            taskset -c "$afinidad" ./unidad-2 &
            pids+=("$!")
        fi
    done

    # Esperar a todos y detectar errores en cualquiera de los procesos.
    local fallo=0
    for pid in "${pids[@]}"; do
        wait "$pid" || fallo=1
    done
    if (( fallo )); then
        echo "Falló un proceso; no se generará un reporte válido." >&2
        exit 1
```

```
    fi

    fin=$(date +%s%N)
    duracion=$((fin - inicio))
}

medir "Secuencial" "$primera_cpu"
secuencial=$duracion
medir "Pseudoparalela" "$primera_cpu"
pseudoparalela=$duracion

paralela=0
if [[ "$cpus" == "$primera_cpu" ]]; then
    echo
    echo "Paralela omitida: sólo hay una CPU lógica permitida."
else
    medir "Paralela" "$cpus"
    paralela=$duracion
fi

echo
echo "=== REPORTE: tiempo total de las 10 tareas ==="
awk -v s="$secuencial" -v p="$pseudoparalela" -v r="$paralela" 'BEGIN {
    printf "%-20s %12s\n", "Modalidad", "Tiempo (s)"
    printf "%-20s %12.3f\n", "Secuencial", s / 1e9
    printf "%-20s %12.3f\n", "Pseudoparalela", p / 1e9
    if (r > 0) {
        printf "%-20s %12.3f\n", "Paralela", r / 1e9
        printf "\nAceleración paralela respecto de secuencial: %.2fx\n", s / r
    } else {
        printf "%-20s %12s\n", "Paralela", "No disponible"
    }
}'
```

Anexo | Código fuente

unidad-3/compilar.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
#!/bin/bash

# Verificar GCC
if ! command -v gcc &> /dev/null; then
    echo "GCC no está instalado. Instalando..."
    sudo dnf install -y gcc
fi

# Compilar
gcc -Wall -Wextra -O0 paginacion.c -o paginacion
if [ $? -ne 0 ]; then
    echo "Error: no se pudo compilar paginacion.c"
    exit 1
fi
chmod +x paginacion

# --- INICIO DEL EJERCICIO PRÁCTICO ---
echo "======================================================="
echo "Escenario: Simulando un pico de consumo de memoria"
echo "de un servicio backend para observar la paginación."
echo "======================================================="

echo -e "\n[Sistema] Memoria antes de la ejecución (free -h):"
free -h

# Ejecutar el programa en C en segundo plano
./paginacion &
C_PID=$!

# Esperar a que el programa en C asigne la memoria (basado en sus sleeps)
sleep 5

echo -e "\n[Sistema] Monitoreando el proceso con PID $C_PID (ps):"
# Mostrar usuario, PID, % de memoria, Tamaño Virtual (VSZ) y Memoria Residente (RSS)
ps -o user,pid,%mem,vsz,rss,comm -p $C_PID

echo -e "\n[Sistema] Estadísticas de memoria virtual (vmstat):"
vmstat 1 3

# Esperar a que el programa en C termine naturalmente
wait $C_PID

echo -e "\n[Sistema] Memoria después de la liberación (free -h):"
free -h
echo "======================================================="
```

Anexo | Código fuente

unidad-3/paginacion.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    size_t total = 1024 * 1024 * 100; // 100 MiB

    printf("[Programa C] PID del proceso: %d\n", getpid());
    printf("[Programa C] Esperando 3 segundos antes de asignar memoria...\n");
    sleep(3);

    char *buf = malloc(total);
    if (buf == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Forzar la asignación real de marcos de página en la RAM
    for (size_t i = 0; i < total; i += 4096)
    {
        buf[i] = 1;
    }

    printf("[Programa C] 100 MiB paginados. Manteniendo memoria ocupada por 10 segundos...\n");
    sleep(10);

    free(buf);
    printf("[Programa C] Memoria liberada. Fin.\n");
    return 0;
}
```

Anexo | Código fuente

unidad-4/unidad-4.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #include <stdio.h>
 2  #include <stdlib.h>
 3
 4  int main() {
 5      printf("Iniciando la configuración del disco de la Unidad 4...\n");
 6
 7      // Ejecuta el script de bash automatizado
 8      int ret = system("./unidad-4.sh");
 9
10      if (ret == -1) {
11          perror("Error al ejecutar el script de configuración");
12          return 1;
13      }
14
15      printf("Configuración de almacenamiento finalizada con éxito.\n");
16      return 0;
17  }
```

Anexo | Código fuente

unidad-4/unidad-4.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #!/bin/bash
 2  DISCO="/dev/vdb"
 3  PARTICION="/dev/vdb1"
 4  PUNTO_MONTAJE="/mnt/datos"
 5
 6  # Crear partición de forma automatizada
 7  echo -e "n\np\n1\n\n\nw" | fdisk "$DISCO"
 8
 9  # Formatear en XFS
10  mkfs.xfs -f "$PARTICION"
11
12  # Crear punto de montaje
13  mkdir -p "$PUNTO_MONTAJE"
14
15  # Obtener UUID y configurar persistencia en /etc/fstab
16  UUID=$(blkid -s UUID -o value "$PARTICION")
17  if [ -z "$UUID" ]; then
18    echo "Error: No se pudo obtener el UUID."
19    exit 1
20  fi

21
22  sed -i "\|$PUNTO_MONTAJE|d" /etc/fstab
23  echo "UUID=$UUID  $PUNTO_MONTAJE  xfs  defaults  0  0" >> /etc/fstab
24
25  # Aplicar y verificar
26  mount -a
27  df -h | grep "$PUNTO_MONTAJE"
28  echo "¡Listo! Disco configurado y montado correctamente."
```

Anexo | Código fuente

unidad-5/crowdstrike.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  /*
 2   * Unidad 5 - Caso real: CrowdStrike (19 de julio de 2024)
 3   *
 4   * Simula el error del "Content Interpreter" de CrowdStrike Falcon:
 5   * el interprete esperaba 20 campos, pero la actualizacion usaba un campo 21.
 6   * Al leer el campo 21 accede fuera del arreglo (out-of-bounds read).
 7   *
 8   * Aca corre en MODO USUARIO: el SO mata solo a este proceso.
 9   * En CrowdStrike corria en MODO KERNEL (era un driver): cayo todo Windows.
10   *
11   * Uso:  ./crowdstrike 21              -> version con el error
12   *       ./crowdstrike 21 --validar    -> version corregida
13   */
14  #include <stdio.h>
15  #include <stdlib.h>
16  #include <string.h>
17  #include <sys/mman.h>
18  #include <unistd.h>
19
20  #define CAMPOS 20   /* el interprete fue programado para 20 campos */

21
22  int main(int argc, char *argv[]) {
23      int pedido = (argc > 1) ? atoi(argv[1]) : 21;
24      int validar = (argc > 2 && strcmp(argv[2], "--validar") == 0);
25
26      /* Se reserva una pagina para los 20 campos y se protege la pagina siguiente,
27         asi leer el campo 21 siempre cae en memoria prohibida (igual que en el caso real) */
28      long pag = sysconf(_SC_PAGESIZE);
29      char *mem = mmap(NULL, 2 * pag, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1,
    0);
30      mprotect(mem + pag, pag, PROT_NONE);
31      const char **campos = (const char **)(mem + pag - CAMPOS * sizeof(char *));
32
33      for (int i = 0; i < CAMPOS; i++) campos[i] = "valor";
34
35      printf("Interprete: espera %d campos. La actualizacion usa %d.\n", CAMPOS, pedido);
36
37      if (validar && pedido > CAMPOS) {
38          printf("ERROR: la actualizacion pide %d campos y solo hay %d. Se rechaza.\n", pedido,
    CAMPOS);

39          printf("El sistema sigue funcionando.\n");
40          return 1;
41      }
42
43      for (int i = 0; i < pedido; i++) {
44          printf("  Leyendo campo %2d: ", i + 1);
45          fflush(stdout);
46          printf("%s\n", campos[i]);   /* campos[20] esta fuera del arreglo */
47      }
48      printf("Actualizacion aplicada.\n");
49      return 0;
50  }
```

Anexo | Código fuente

unidad-5/crowdstrike.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #!/bin/bash
 2  cd -- "$(dirname -- "$0")"
 3
 4  gcc -O0 crowdstrike.c -o crowdstrike
 5
 6  echo "=== 1. Los drivers corren dentro del kernel ==="
 7  echo "Módulos (drivers) cargados en el kernel: $(lsmod | tail -n +2 | wc -l)"
 8  modinfo virtio_blk 2>/dev/null | grep -E '^(filename|description)' || lsmod | head -5
 9  echo "El sistema está funcionando desde: $(uptime -p)"
10
11  echo
12  echo "=== 2. El error de CrowdStrike, simulado en MODO USUARIO ==="
13  ./crowdstrike 21
14  echo "Código de salida: $?   (139 = el SO mató al proceso con SIGSEGV)"
15
16  echo
17  echo "=== 3. El sistema operativo sigue funcionando ==="
18  echo "Uptime: $(uptime -p)"
19  sudo dmesg 2>/dev/null | grep -i "crowdstrike.*segfault" | tail -1
20

21  echo
22  echo "=== 4. La versión corregida valida la cantidad de campos ==="
23  ./crowdstrike 21 --validar || true
```

Anexo | Código fuente

unidad-5/ejecutar.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #!/bin/bash
 2  set -e
 3
 4  cd -- "$(dirname -- "$0")"
 5
 6  # Instalar GCC si no está instalado.
 7  if ! command -v gcc &> /dev/null; then
 8      echo "GCC no está instalado. Instalando..."
 9      sudo dnf install -y gcc
10  fi
11
12  gcc -O0 unidad-5.c -o unidad-5
13
14  # Disco principal del sistema (vda, sda o nvme0n1).
15  DISCO=/dev/$(lsblk -dno NAME,TYPE | awk '$2=="disk" && $1 !~ /^(zram|loop)/{print $1; exit}')
16
17  # Archivo de prueba de 2 MB.
18  head -c 2000000 /dev/urandom > prueba.bin
19
20  echo "=== Programa: tipo de dispositivo y uso de búfer ==="

21  ./unidad-5 prueba.bin "$DISCO" /dev/null /dev/tty /dev/urandom
22
23  echo
24  echo "=== Número mayor -> driver (/proc/devices) ==="
25  MAYOR=$((16#$(stat -c '%t' "$DISCO")))
26  grep -E "^ *($MAYOR|1|5) " /proc/devices
27
28  echo
29  echo "=== Interrupciones del controlador de disco antes y después de escribir 100 MB ==="
30  contar() { grep -Ei "virtio|ahci|nvme|ata" /proc/interrupts | awk '{for(i=2;i<=NF;i++) if($i ~
     /^[0-9]+$/) s+=$i} END{print s+0}'; }
31  ANTES=$(contar)
32  dd if=/dev/zero of=escritura.tmp bs=1M count=100 oflag=direct status=none
33  DESPUES=$(contar)
34  rm -f escritura.tmp
35  echo "Antes: $ANTES   Después: $DESPUES   Nuevas: $((DESPUES - ANTES))"
```

Anexo | Código fuente

unidad-5/unidad-5.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/sysmacros.h>
#include <time.h>
#include <unistd.h>

static double ahora(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s <archivo> [dispositivos...]\n", argv[0]);
        printf("Ej:  %s prueba.bin /dev/vda /dev/null /dev/tty /dev/urandom\n", argv[0]);
        return 1;
    }

    /* Parte 1: tipo de dispositivo y numeros mayor/menor */
    for (int i = 2; i < argc; i++) {
        struct stat st;
        if (stat(argv[i], &st) < 0) { perror(argv[i]); continue; }
        printf("%-14s %-9s mayor=%-3u menor=%u\n", argv[i],
               S_ISBLK(st.st_mode) ? "BLOQUE" : S_ISCHR(st.st_mode) ? "CARACTER" : "ARCHIVO",
               major(st.st_rdev), minor(st.st_rdev));
    }

    /* Parte 2: leer el mismo archivo con distintos tamanos de bufer */
    printf("\nBufer(B)  Llamadas read()  Tiempo\n");
    int tams[] = {1, 512, 4096, 65536};
    for (int t = 0; t < 4; t++) {
        char *buf = malloc(tams[t]);
        int fd = open(argv[1], O_RDONLY);
        if (fd < 0) { perror(argv[1]); return 1; }
        long llamadas = 0;
        double t0 = ahora();
        while (read(fd, buf, tams[t]) > 0) llamadas++;

        printf("%-9d %-16ld %.4f s\n", tams[t], llamadas, ahora() - t0);
        close(fd);
        free(buf);
    }
    return 0;
}
```

Anexo | Código fuente

unidad-6/unidad-6.c

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    char datos[2][16] = {"usuario", "clave123"};
    int campo, longitud;

    printf("Campo a consultar (0: usuario, 1: contraseña): ");
    if (scanf("%d", &campo) != 1) {
        return 1;
    }

    if (campo == 1) {
        printf("Error: no tenés permiso para leer la contraseña.\n");
        return 1;
    }

    if (campo != 0) {
        printf("Error: campo inexistente.\n");
        return 1;

    }

    printf("Longitud a leer: ");
    if (scanf("%d", &longitud) != 1) {
        return 1;
    }

    // Evita salir del bloque completo, pero no protege el campo privado.
    if (longitud < 1 || longitud > (int) sizeof(datos)) {
        printf("Error: longitud inválida.\n");
        return 1;
    }

    printf("Resultado: ");
    fflush(stdout);

    // Falla: el campo está autorizado, pero la longitud puede abarcar ambos.
    write(STDOUT_FILENO, &datos, (size_t) longitud);
    write(STDOUT_FILENO, "\n", 1);


    return 0;
}
```

Anexo | Código fuente

unidad-6/unidad-6.sh

Código del repositorio. Numeración añadida; las líneas largas se continúan con sangría.

```
 1  #!/bin/bash
 2  set -e
 3
 4  # Trabajar desde la carpeta del script.
 5  cd -- "$(dirname -- "$0")"
 6
 7  # Instalar GCC si no está instalado.
 8  if ! command -v gcc &> /dev/null; then
 9      echo "GCC no está instalado. Instalando..."
10      sudo dnf install -y gcc
11  fi
12
13  # Compilar el programa C. Si falla, no ejecutar un binario anterior.
14  gcc -Wall -Wextra -O0 unidad-6.c -o unidad-6
15
16  # Ejecutar el programa compilado.
17  ./unidad-6
```

Referencia de versión (control documental)

Commit: 5936daac62e70f03619d077531353763733f7bfa

fix pdf style

Fecha del commit: 2026-11-02 23:29:19 -0300 | Rama: main